



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 10

Project Title: Priority Scheduling System

Team Members

Roll Number	Student Name	Signature
2520030067	KODEBOINA PRIYADARSHINI	
2520030108	GANDLA SAHASRA	
2520030400	CHEETI SRIHITHA	

1. Abstract

The **Priority Scheduling System** project demonstrates how operating systems manage process execution by assigning and enforcing priority levels. Each process is given a priority value, and the CPU is allocated to the process with the highest priority. The project explores both **preemptive** and **non-preemptive** scheduling modes, showing how high-priority tasks can interrupt or precede lower-priority ones to ensure responsiveness.

Key operating system concepts such as **process creation, process identification, scheduling queues, turnaround time, waiting time, and response time** are addressed. The system highlights challenges like **starvation** of low-priority processes and integrates solutions such as **aging**, which gradually increases the priority of waiting tasks to maintain fairness.

The implementation involves a simulation program that models process arrival, burst time, and priority assignment, producing outputs such as **Gantt charts, completion times, and average performance metrics**. The expected outcome is a functional demonstration of how priority scheduling improves responsiveness for critical tasks while balancing efficiency and fairness.

This project strengthens understanding of **CPU scheduling algorithms**, provides hands-on experience with process management, and illustrates the importance of scheduling in both academic learning and real-time system applications.

2. Problem Statement

In an operating system, many processes run at the same time and compete for CPU resources. If all processes are treated equally, important tasks may get delayed, affecting system performance and responsiveness. The problem is to design a scheduling system that gives preference to **high-priority processes** while still allowing lower-priority ones to execute fairly. Without such a mechanism, low-priority tasks may face **starvation**, and the CPU may not be used efficiently. This project aims to implement a **Priority Scheduling System** that selects processes based on their priority values, supports both **preemptive** and **non-preemptive** scheduling, and uses techniques like **aging** to avoid starvation. The solution will demonstrate how priority scheduling improves responsiveness, fairness, and efficiency in process management.

3. Objectives

- To understand the concept of **CPU scheduling** using priority levels.
- To design and implement both **preemptive** and **non-preemptive** priority scheduling algorithms.
- To simulate process execution with attributes such as **arrival time, burst time, and priority value**.
- To calculate and analyze **waiting time, turnaround time, and response time** for scheduled processes.
- To address challenges like **starvation** and apply techniques such as **aging** for fairness.
- To demonstrate scheduling results using **Gantt charts and performance metrics**.
- To strengthen knowledge of **operating system process management** through practical implementation.

4. Proposed Methodology

The project will be designed and implemented as a **Linux-based C/C++ application** that demonstrates scheduling of processes based on assigned priority values.

- **Process Creation:** Define processes with attributes – ID, Arrival Time, Burst Time, Priority.
- **Process Identification:** Assign unique PID and store in the ready queue.
- **Scheduling Algorithm:** Implement **Preemptive** and **Non-Preemptive** priority scheduling.
- **Priority Handling:** CPU selects highest priority; ties resolved with **FCFS**.
- **Starvation Control:** Apply **aging** to gradually raise waiting process priority.
- **Execution Flow:** Simulate scheduling, generate **Gantt charts**, and calculate metrics.
- **Performance Evaluation:** Compare results with other algorithms for fairness and efficiency.
- **Testing:** Run in Linux environment with varied process sets to verify correctness.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – fork()	Creates child processes to simulate scheduling.
Process Identification – getpid(), getppid()	Identifies processes for management in the queue.

Scheduling – Priority Algorithm	Allocates CPU based on priority values.
Priority Handling – FCFS	Resolves ties when processes have equal priority.
Starvation Control – Aging	Prevents low-priority processes from waiting forever.
Performance Metrics – Calculation	Computes waiting time, turnaround time, response time.
Visualization – Gantt Chart	Shows execution order and scheduling results.
Testing – Linux Execution	Runs with different process sets to check fairness.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030067	KODEBOINA PRIYADARSHINI	
2520030108	GANDLA SAHASRA	
2520030400	CHEETI SRIHITHA	

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the environment to develop, compile, and run program.
C / C++	Used to implement process creation and priority scheduling logic.
GCC / G++ Compiler	Compiles the C/C++ source code into an executable program.
POSIX Libraries	Offer APIs for process management and system calls.
Linux Terminal / Shell	Used to compile, execute, and test the scheduling application.

8. Expected Outcome

The project is expected to produce a functional **Linux-based C/C++ application** that demonstrates how processes are scheduled based on assigned priority values.

- The program will create and manage processes with **ID, arrival time, burst time, and priority**.
- CPU will schedule processes using **Preemptive** and **Non-Preemptive Priority Scheduling**.
- **FCFS** will resolve ties, and **aging** will prevent starvation.
- Results will show **Gantt charts, waiting time, turnaround time, and response time**.
- The system will demonstrate **fairness, efficiency, and responsiveness** in scheduling.

9. GitHub Repository

GitHub Repository Link:

Repository Created:

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Y. Harika Devi

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: